Priority Rule Based Heuristic – RCPSP

Project-aligned implementation using Highest Number of Successors (HNS) and the Serial Scheduling Generation Scheme (SSGS). The exact source code below was executed successfully before creating this PDF.

```
# Priority Rule Based Heuristic for RCPSP
# Project: AI-Enhanced Metaheuristics Control for Project Scheduling
# Rule: Highest Number of Successors (HNS)
# Schedule generation: Serial Scheduling Generation Scheme (SSGS)

TASKS = {
    1: (4, 2, []),
    2: (3, 2, [1]),
    3: (5, 3, [1]),
    4: (2, 2, [2]),
    5: (4, 2, [2]),
    6: (3, 2, [3]),
    7: (2, 3, [4, 6]),
    8: (1, 1, [5, 7])
}

RESOURCE_CAPACITY = 5


def successor_count(task):
    """Return the total number of downstream tasks."""
    successors = set()

    changed = True
    while changed:
        changed = False

        for t, (_, _, predecessors) in TASKS.items():
            if t in successors:
                continue

            if t != task and task in predecessors:
                successors.add(t)
                changed = True

            elif any(p in successors for p in predecessors):
                successors.add(t)
                changed = True

    return len(successors)


def priority_order(ready_tasks):
    """HNS: most downstream successors gets highest priority."""
    return sorted(
        ready_tasks,
        key=lambda t: (
            -successor_count(t),
            TASKS[t][0],  # shorter duration as tie-breaker
            t
        )
    )


def schedule_project():
    """SSGS: schedule each ready task at its earliest feasible time."""
    unscheduled = set(TASKS)
    schedule = {}

    while unscheduled:
        ready = [
            t for t in unscheduled
            if all(p in schedule for p in TASKS[t][2])
        ]

        if not ready:
            raise ValueError("Invalid precedence graph: cycle detected.")

        task = priority_order(ready)[0]

        duration, demand, predecessors = TASKS[task]

        earliest = 0
        if predecessors:
            earliest = max(schedule[p]["finish"] for p in predecessors)

        start = earliest

        while True:
            finish = start + duration
            feasible = True
```

```
            for time in range(start, finish):
                used = sum(
                    x["resource"]
                    for x in schedule.values()
                    if x["start"] <= time < x["finish"]
                )

                if used + demand > RESOURCE_CAPACITY:
                    feasible = False
                    break

            if feasible:
                break

            start += 1

        schedule[task] = {
            "start": start,
            "finish": finish,
            "duration": duration,
            "resource": demand,
            "priority": successor_count(task)
        }

        unscheduled.remove(task)

    return schedule


def main():
    schedule = schedule_project()
    makespan = max(x["finish"] for x in schedule.values())

    print("\nPRIORITY RULE BASED RCPSP HEURISTIC")
    print("=" * 65)
    print("Priority rule : Highest Number of Successors (HNS)")
    print("Schedule      : Serial Scheduling Generation Scheme (SSGS)")
    print("Resource limit:", RESOURCE_CAPACITY)

    print("\nTask | Priority | Start | Finish | Duration | Resource")
    print("-" * 65)

    for task in sorted(schedule):
        x = schedule[task]
        print(
            f"{task:4} | {x['priority']:8} | "
            f"{x['start']:5} | {x['finish']:6} | "
            f"{x['duration']:8} | {x['resource']:8}"
        )

    print("-" * 65)
    print("Project makespan:", makespan)


if __name__ == "__main__":
    main()
```